PRAKTIKUM SISTEM OPERASI

MODUL 6

PROSES SEKUENSIAL DAN KONKUREN

Oleh:

Aulia Jasifa Br Ginting 2311104060

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

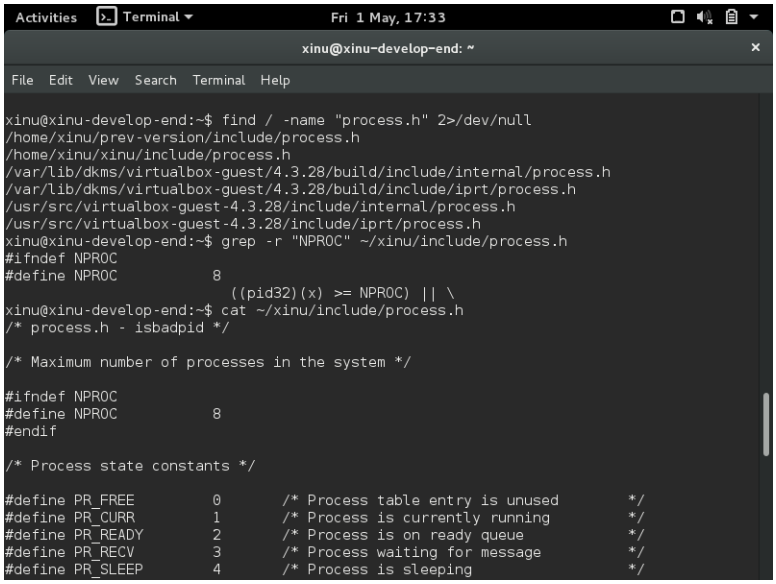
UNIVERSITAS TELKOM

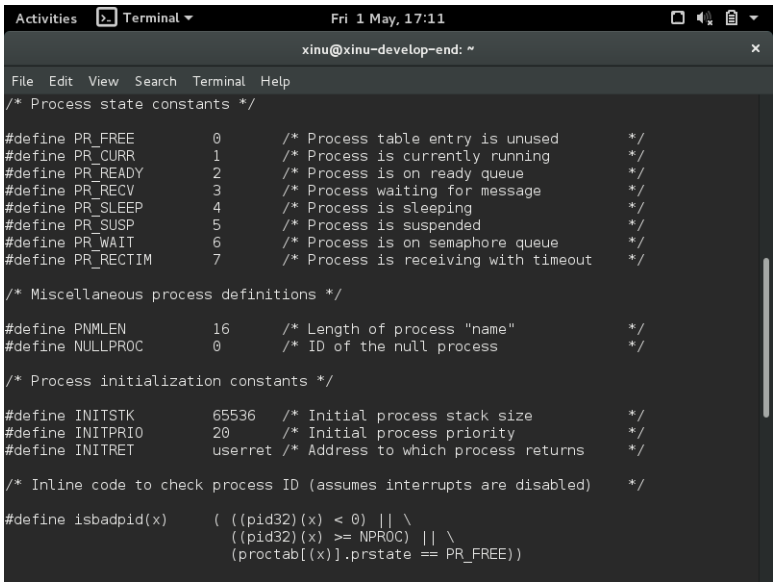
2026

JURNAL/TP

1. Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah $cat /proc/sys/kernel/pid_max

   - File process.h





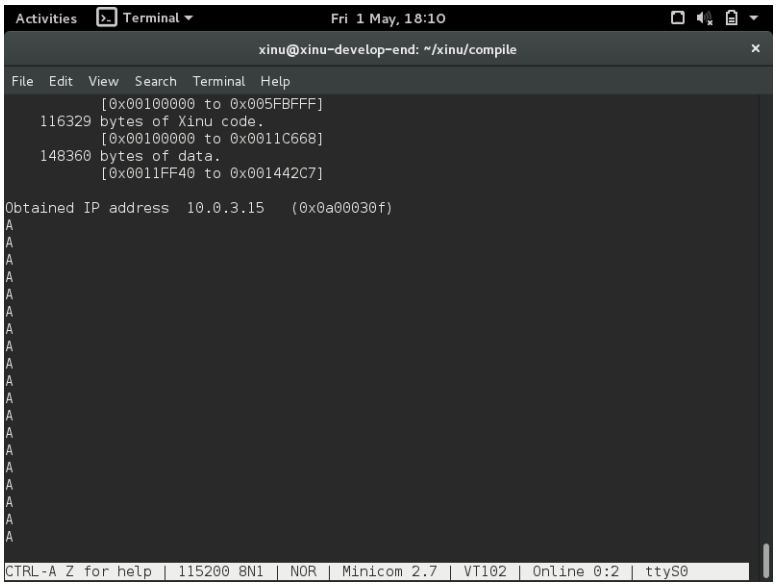
- Mengubah NPROC dari 8 ke 150

```
Activities   Terminal   Fri 1 May, 17:34
xinu@xinu-develop-end: ~
File  Edit  View  Search  Terminal  Help
  GNU nano 2.2.6        File: /home/xinu/xinu/include/process.h

/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC           8
#endif

/* Process state constants */

#define PR_FREE         0       /* Process table entry is unused        */
#define PR_CURR         1       /* Process is currently running         */
#define PR_READY        2       /* Process is on ready queue            */
#define PR_RECV         3       /* Process waiting for message          */
#define PR_SLEEP        4       /* Process is sleeping                  */
#define PR_SUSP         5       /* Process is suspended                 */
#define PR_WAIT         6       /* Process is on semaphore queue        */
#define PR_RECTIM       7       /* Process is receiving with timeout    */

/* Miscellaneous process definitions */

#define PNMLEN          16      /* Length of process "name"             */
#define NULLPROC        0       /* ID of the null process               */

^G Get Help   ^O WriteOut   ^R Read File   ^Y Prev Page   ^K Cut Text   ^C Cur Pos
^X Exit       ^J Justify    ^W Where Is    ^V Next Page   ^U UnCut Text ^T To Spell
```

```
        umsg32  prmsg;          /* Message sent to this process         */
        bool8   prhasmsg;       /* Nonzero iff msg is valid             */
        int16   prdesc[NDESC];  /* Device descriptors for process       */
};

/* Marker for the top of a process stack (used to help detect overflow) */
#define STACKMAGIC      0x0A0AAAA9

extern  struct  procent proctab[];
extern  int32   prcount;        /* Currently active processes           */
extern  pid32   currpid;        /* Currently executing process          */
xinu@xinu-develop-end:~$ nano /home/xinu/xinu/include/process.h
xinu@xinu-develop-end:~$ grep -n "NPROC" ~/xinu/include/process.h
5:#ifndef NPROC
6:#define       NPROC           150
34:                     ((pid32)(x) >= NPROC) || \
xinu@xinu-develop-end:~$
```

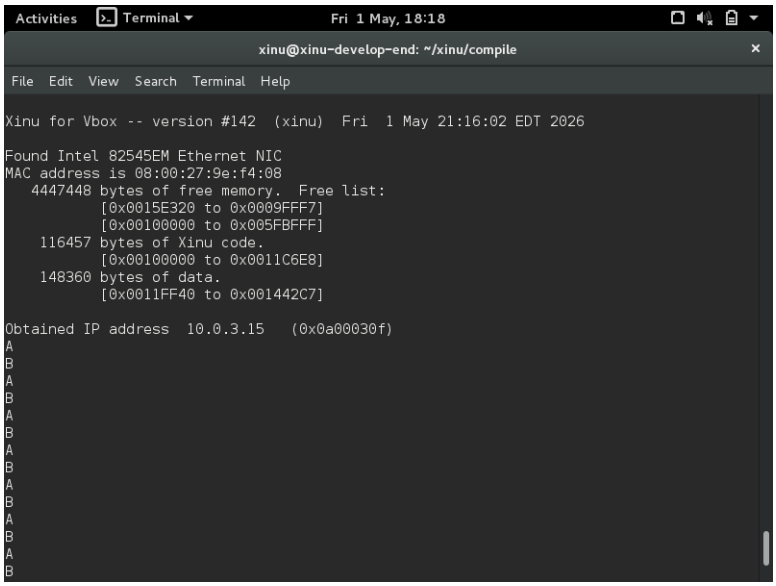
2. Jalankan kode main_ex1!

```c
/* main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
        sndA();
        sndB();
        return OK;
}

void sndA(void)
{
        while(1){
        printf("A\n");
        sleepms(1000);
        }
}

void sndB(void)
{
        while(1){
        printf("B\n");
        sleepms(1000);
        }
}
```



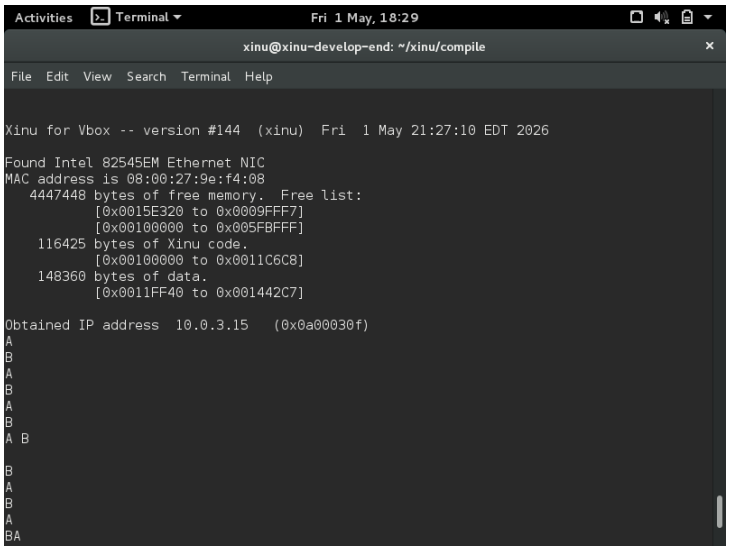
3. Jalankan kode main_ex2!

```c
/* main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
        resume(create(sndA, 1024, 20, "process A", 0));
        resume(create(sndB, 1024, 20, "process B", 0));
        return OK;
}

void sndA(void)
{
        while(1){
        printf("A\n");
        sleepms(1000);
        }
}

void sndB(void)
{
        while(1){
        printf("B\n");
        sleepms(1000);
        }
}
```



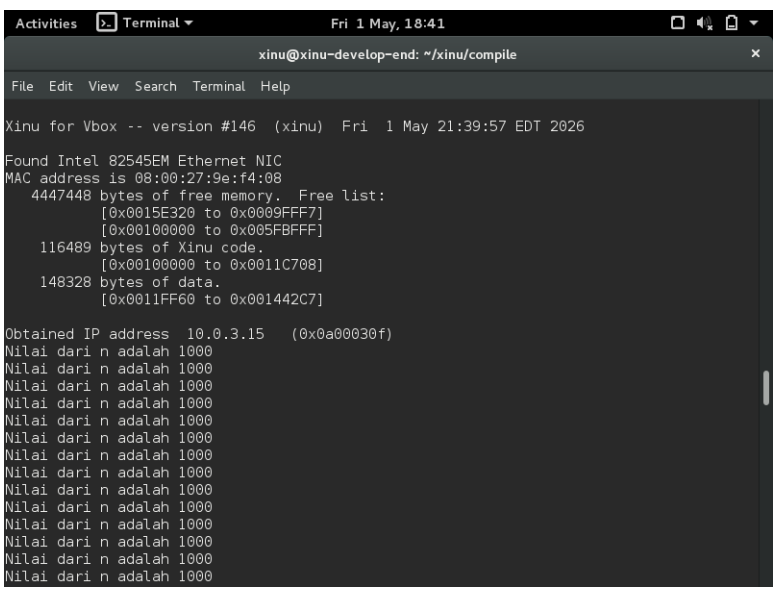
4. Jalankan kode main_ex3!

```c
/* main.c - main*/

#include <xinu.h>
void sndChar(char);

process main(void)
{
        resume(create(sndChar, 1024, 20, "Send A", 1, 'A'));
        resume(create(sndChar, 1024, 20, "Send B", 1, 'B'));
        return OK;
}

void sndChar(char c)
{
        while(1){
        printf("%c \n", c);
        sleepms(1000);
        }
}
```



5. Buatlah 2 proses produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```c
/* main.c - main*/

#include <xinu.h>

int32 n = 0;

void produser(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                n++;
        }
}

void konsumer(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                printf("Nilai dari n adalah %d\n", n);
        }
}

process main(void) {
        resume(create(produser, 1024, 20, "producer", 0));
        resume(create(konsumer, 1024, 20, "consumer", 0));
        return OK;
}
```



Hasil program menunjukkan bahwa nilai n selalu 1000 karena proses produser menyelesaikan seluruh iterasi terlebih dahulu sebelum proses konsumer dijalankan. Hal ini terjadi karena tidak adanya delay atau mekanisme pembagian waktu yang memaksa kedua proses berjalan secara bergantian. Meskipun demikian, program ini

tetap memiliki potensi race condition karena variabel n diakses secara bersama tanpa sinkronisasi. Jika penjadwalan proses berbeda, maka hasil yang ditampilkan dapat menjadi tidak terurut atau tidak konsisten.